

## ① Prac Final

- all students w/ GPA  $\geq 3.5$

```
SELECT * FROM students
WHERE gpa >= 3.5
```

$\sigma_{gpa \geq 3.5}$  (students)

- customers (id, name)
- orders (id, customer\_id, amount)

→ join customers w/ their orders  
(only customers who have orders)

a) `SELECT * FROM customers AS c, orders AS o  
WHERE c.id = o.customer_id;`

b) `SELECT * FROM customers AS c  
INNER JOIN orders AS o  
ON c.id = o.customer_id;`

2 join options

\* NOT LEFT JOIN b/c we don't want all customers

customers  $\bowtie$  customers.id = orders.customer\_id (orders)

- Students (id, name, email) & email has NULLS.

→ count how many students have non-NULL email

`SELECT COUNT(email) FROM students;` exclude null

`COUNT(*)` include nulls + duplicates

`COUNT(DISTINCT email)` unique only

## True/False

- Either network under heavy load or system failure  
If system failure, notify incident response.

→ Engineer confirm system is NOT under heavy load.  
Then system must have notified incident response.

True      Not H means F.  $F \rightarrow N$

- Given:  $(P \rightarrow Q) \wedge (R \rightarrow Q)$  then  $P \rightarrow R$       True  $\downarrow$

| P | Q | R | $\neg Q$ | $\neg R$ | $P \rightarrow Q$ | $\neg P$ | $R \rightarrow \neg Q$ | $P \rightarrow \neg R$ | $\neg(P \rightarrow R)$ |
|---|---|---|----------|----------|-------------------|----------|------------------------|------------------------|-------------------------|
| T | T | T | F        | F        | T                 | F        | F                      | F                      | T                       |
| T | T | F | T        | T        | F                 | F        | T                      | T                      | F                       |
| T | F | T | T        | F        | F                 | T        | T                      | T                      | F                       |
| T | F | F | T        | T        | F                 | T        | T                      | T                      | F                       |
| F | T | T | F        | F        | T                 | F        | F                      | T                      | T                       |
| F | T | F | T        | T        | T                 | T        | T                      | T                      | T                       |
| F | F | T | T        | F        | T                 | T        | T                      | T                      | T                       |
| F | F | F | T        | T        | T                 | T        | T                      | T                      | T                       |

$R \rightarrow \neg Q \leftrightarrow Q \rightarrow \neg R$   
 $(P \rightarrow Q) \wedge (Q \rightarrow \neg R) \Rightarrow P \rightarrow \neg R$

## Relational Algebra

- Student (SID, Lname, Fname, MI, GPA, Major)
- Course (CID, Cname, Dept)
- Grade (SID, CID, semester, Grade)

Find all course names + semester where Oski Bear got grade 'A'

$OB\_SID \leftarrow \pi_{SID} (\sigma_{Lname='Bear' \wedge Fname='OSKI'} (Student))$

$A\_GRADE \leftarrow \sigma_{grade='A'} (Grade)$

$OB\_TAKEN \leftarrow OB\_SID \bowtie A\_GRADE$

$R \leftarrow \pi_{Cname, semester} (OB\_TAKEN \bowtie Course)$

## ② Relational Alg

- Projection ( $\pi$ ) = pick cols

$\pi_{id, citation} (Stops)$

- Selection ( $\sigma$ ) = filter rows by cond.

$\pi_{id, citation} (\sigma_{age \leq 50} (Stops))$

- renaming ( $\rho$ )

$\rho_{stops\_data (id \rightarrow personid) (Stops)}$

- Cartesian product ( $\bowtie$ )

↳ merge col from 2 rel + pairwise

$\sigma_{stops.location = Zips.location} (Stops \times Zips)$

- Theta Join ( $\bowtie_{\theta}$ )

↳ Cartesian w/ select cond.

a  $\bowtie_{a.col2 > b.col2}$  b

- Equi Join

↳ Theta w/ equality cond

$stops \bowtie_{stops.location = Zips.location} (Zips)$

- Natural Join

↳ 2 relations share attr name + type

$stops \bowtie_{\sim} Zips$

- Left/Right Outer Join

people  $\bowtie_{\sim}$  menu / people  $\bowtie_{\sim}$  menu

- grouping/agg

↳  $\gamma_{groupattr; aggfunc(attr) \rightarrow newName}(R)$

$\gamma_{major, count(sid) \rightarrow numstudents}(Student)$

Find student SID who got 'A' in CID = 'CS101'

$R \leftarrow \pi_{sid} (\sigma_{grade='A' \wedge CID='CS101'} (Grade))$

First, last name of everyone who took

'intro DB' in Fall 2024

$IDB \leftarrow \pi_{CID} (\sigma_{Cname='intro DB'} (Course))$

$FS \leftarrow \sigma_{semester='Fall 2024'} (Grade)$

$DB\_IN\_FALL \leftarrow IDB \bowtie_{CID=CID} FS$

$R \leftarrow \pi_{Lname, Fname} (DB\_IN\_FALL \bowtie_{sid=sid} Student)$

## ③ SQL

- Add/drop column

`ALTER TABLE [tablename]`

`PROP/ADD/MODIFY COLUMN [column_name][datatype];`

- Delete

↳ delete data from a table referenced by another table w/ a FK

`ON DELETE CASCADE`

- by condition `DELETE FROM [table_name]`

`WHERE [condition]` ← ignore to delete All

- Update

`UPDATE [table_name]`

`SET [condition]`

`WHERE [condition]`

- UNION (All = duplicates)

- combine 2+ select statements

- of same # cols + data-types

- arithmetic on data in columns (price + 0.1) As Discount

- between

`WHERE orderDate BETWEEN '1997-07-04'`  
`AND '1996-08-04'`

- inclusive range [a, b]

- ORDER BY [colname] DESC/ASC

- Nested Queries

`SELECT o.orderID  
FROM orders AS o  
WHERE o.customerID IN (  
SELECT c.customerID  
FROM customer AS c  
WHERE c.customerName = 'Centro');`

- Joins

↳ implicit inner/natural  
`SELECT Fname, Lname, o.orderID, o.orderDate  
FROM Employee e, orders o  
WHERE e.EmployeeID = o.employeeID`  
(rows in both tables)

- SELECT [cols]

`FROM [table t1]  
LEFT/RIGHT/INNER/OUTER JOIN [table t2]  
ON [condition] = cond;`

↳ all rows on left kept

- Agg functions

- count, avg, min, max, stddev, variance, sum

- GROUP BY [col-names to be grouped by]

- HAVING [condition]

- if col not in group by, add to SELECT alone

EX2: `SELECT p.prodName  
FROM prod p, orderdetails od, orders o  
WHERE p.prodID = od.prodID AND od.orderID = o.orderID  
GROUP BY p.prodName  
HAVING COUNT(DISTINCT o.cid) >= 2`

- DROP TABLE IF EXISTS [table.name];

- FK/PK

`CREATE TABLE product (`

`ProductID INTEGER PRIMARY KEY`

`AUTO_INCREMENT,`

`SupplierID INTEGER`

`FOREIGN KEY (SupplierID) REFERENCES`

`supplier(supplierID)`

- Add row

`INSERT INTO category VALUES (1, ...);`

- Delete row

`DELETE FROM category  
WHERE categoryID = 2`

- UPDATE

`UPDATE category  
SET description = 'Tea, Soda'`

`WHERE categoryID = 2`

- Rename

`ALTER TABLE product  
RENAME COLUMN Price TO unitPrice`

- SELECT-FROM-WHERE

- WHERE COND: =, <=, >=, !=, NULL, IS NOT NULL

`WHERE country IN ('USA', 'CH')`  
or `LIKE "T%"`

- NOT → AND → OR



### ③ Normalization

#### 1NF

- R is in 1NF if all attributes are single-valued

$R(SID, CID, semester, title, slname, Major, Advisor, FacID, FacLname, classroom, OptionalTexts, Grade)$  } UNF

- \* OptionalTexts = sets # atomic = violate 1NF

- Normalize into 2

- 1)  $R_1(SID, CID, semester, title, slname, Major, Advisor, FacID, FacLname, classroom, Grade)$
- 2) OptionalTextbooks ( $CID, semester, Textbook$ )

#### 2NF

- relevant when R has composite key  
(CK = set of attributes to uniquely identify)
- all non-prime attributes must depend on the whole key only
- R is NOT in 2NF if some proper subset of a CK determines a non-prime attribute
- ↳ non-prime attributes not in any CK

$R_1(SID, CID, semester, title, slname, Major, Advisor, FacID, FacLname, classroom, Grade)$

#### Problems

- insert anomalies = can't store FacID, FacLname, etc. w/o students signing up first
- deletion anomalies = lose faculty info if we del. student
- update anomalies = title, FacID, FacLname, classroom repeats
- if classroom changes, every record referring to cid/sem must be updated

- many non-key attributes don't depend on the whole key

| Non-prime attr. | Depends on    | Why partial dependent? |
|-----------------|---------------|------------------------|
| slname          | SID           | partial of CK          |
| Major           | SID           |                        |
| Advisor         | SID           |                        |
| title           | CID           |                        |
| FacID           | CID, semester | partial of CK          |
| FacLname        | CID, semester |                        |
| classroom       | CID, semester |                        |

- only grade is not partially dependent

- Normalize R into 2NF

1. StudentName ( $SID, slname$ )
2. StudentMajor ( $SID, Major, Advisor$ )
3. Course ( $CID, title$ )
4. Cprof ( $CID, semester, FacID, FacLname$ )
5. Croom ( $CID, semester, classroom$ )
6. Grade ( $SID, CID, semester, Grade$ )

$R(A, B, C, D)$

$A \rightarrow C \times AB \rightarrow C \checkmark$   
 $B \rightarrow C \times$

#### 3NF

- a non-prime attribute can't depend on another non-prime attribute
- R is not in 3NF if  $\exists$  nonprime pair  $A, B: A \rightarrow B$
- ⇒ R is 3NF if  $\forall$  FD:  $X \rightarrow Y$  satisfies:
  - 1) X is a candidate key OR
  - 2) Y is a prime attribute

4) Cprof ( $CID, semester, FacID, FacLname$ )

#### Problems

- can't store FacLname w/o course
- FacLname repeated every cid/semester
- If prof lname changes, all will change
- ⇒ Transitive dependency
- $\{CID, semester\} \rightarrow FacID$  and  $FacID \rightarrow FacLname$

- ⇒ normalize

- 4a) CprofName ( $FacID, FacLname$ )
- 4b) CprofID ( $CID, semester, FacID$ )

#### BCNF

- if  $\forall X \rightarrow Y$ , then X is a super-key [candidate key]
- BCNF  $\rightarrow$  3NF  $\rightarrow$  2NF  $\rightarrow$  1NF

### ④ FRQ

#### Enrollment\_UNF:

| StudentID | StudentName | MajorID | MajorName | Courses                                           |
|-----------|-------------|---------|-----------|---------------------------------------------------|
| 1001      | Alice       | M01     | CS        | ((CS101, 'Intro DB', A), (CS102, 'Algo', B+))     |
| 1002      | Bob         | M02     | EE        | ((EE101, 'Circuits', B), (CS101, 'Intro DB', A-)) |
| 1003      | Carol       | M01     | CS        | ((CS101, 'Intro DB', B+), (MATH201, 'Calc', A))   |

#### Assume FD

- StudentID  $\rightarrow$  StudentName, MajorID
- MajorID  $\rightarrow$  MajorName
- CourseID  $\rightarrow$  CourseName
- StudentID, CourseID  $\rightarrow$  Grade (one grade per student-course pair)

UNF = potential attributes to go in PK

- Transform Enrollment\_UNF in 1NF. Show rows + underline PK

| StudentID | StudentName | MajorID | MajorName | <u>CourseID</u> | CourseName | Grade |
|-----------|-------------|---------|-----------|-----------------|------------|-------|
| 1001      | Alice       | M01     | CS        | CS101           | Intro DB   | A     |
| 1001      | Alice       | M01     | CS        | CS102           | Algo       | B+    |
| 1002      | Bob         | M02     | EE        | EE101           | Circuits   | B     |
| 1002      | Bob         | M02     | EE        | CS101           | Intro DB   | A-    |
| 1003      | Carol       | M01     | CS        | CS101           | Intro DB   | B+    |
| 1003      | Carol       | M01     | CS        | MATH201         | Calc       | A     |

- From 1NF, produce 2NF + say which partial dependencies you removed

| student | StudentID | StudentName | MajorID | MajorName |
|---------|-----------|-------------|---------|-----------|
|         | 1001      | Alice       | M01     | CS        |
|         | 1002      | Bob         | M02     | EE        |
|         | 1003      | Carol       | M01     | CS        |

| <u>Course</u> | <u>CourseID</u> | <u>CourseName</u> |
|---------------|-----------------|-------------------|
|               | CS101           | Intro DB          |
|               | CS102           | Algo              |
|               | EE101           | Circuit           |
|               | MATH201         | Calc              |

| Enrollment | StudentID | CourseID | Grade |
|------------|-----------|----------|-------|
|            | 1001      | CS101    | A     |
|            | 1001      | CS 102   | B+    |
|            | 1002      | EE 101   | B     |
|            | 1002      | CS 101   | A-    |
|            | 1003      | CS 101   | B+    |
|            | 1003      | MATH 201 | A     |

- removed partial dependencies:

- StudentID  $\rightarrow$  StudentName
- StudentID  $\rightarrow$  MajorID
- CourseID  $\rightarrow$  CourseName
- Note: I kept (StudentID, CourseID  $\rightarrow$  Grade) b/c not partial
- MajorID  $\rightarrow$  MajorName (transitive)

- Verify if 2NF tables are 3NF. (by identifying at least one transitive dependency) \* NOT 3NF w/ transitive dep.
- If it isn't, decompose into 3NF relations, giving each relation its PK. Show rows + PK

- StudentID  $\rightarrow$  MajorID  $\rightarrow$  MajorName (transitive)

| StudentID | StudentName | MajorID |
|-----------|-------------|---------|
| 1001      | Alice       | M01     |
| 1002      | Bob         | M02     |
| 1003      | Carol       | M01     |

| MajorID | MajorName |
|---------|-----------|
| M01     | CS        |
| M02     | EE        |

- course + enrollment tables are 3NF  $\rightarrow$  depend on keys only

CS

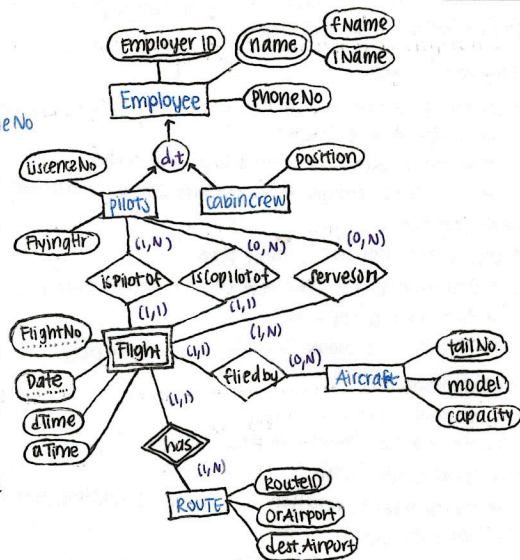
EE

anvi, Catherine, Jose, Kanna, Richard, Sena, Tam, Andre

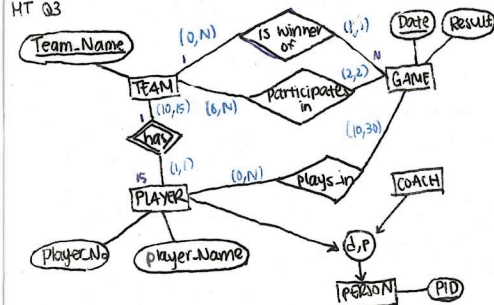


## MT Q2 Airline DB

1. fleet of Aircraft, identified by unique tailNo. w/ model & capacity
2. Employers diff types of Employees w/ unique employerID, name (composed of firstN, lastN), phoneNo
  - either Pilots/cabin crew → specialization
  - pilots store licenseNo, flyingHours
  - cabin crew stores position
3. operates routes w/ unique routeID & connects to specific originAirport & destinationAirport
4. operates flights, identified by flightNo. that is unique only w/in its route, & date. Each flight has departureTime, arrTime
  - ea. flight assigned one aircraft, used for many flights or grounded
  - ea. flight assigned one pilot as captain & one as co-pilot. Pilot can have many flights
  - ea. flight staffed by several cabin crew, which can be assigned to many flights, & at least one cabin crew assigned



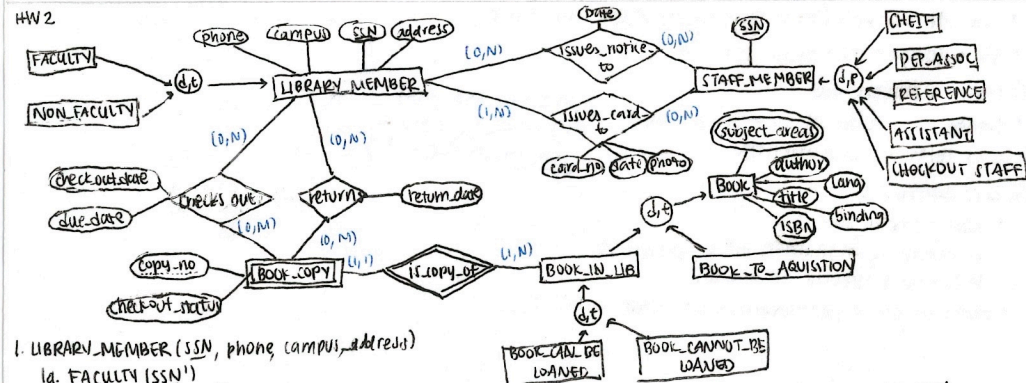
## HT Q3



1. Person (PID)
  - a. coach (PID<sup>1</sup>)
  - b. player (PID<sup>1</sup>, player-No, player-Name, team-name<sup>2</sup>)
2. Game (Date, result | win-team-Name<sup>3</sup>)
3. Team (Team-Name)
4. plays in (GameDate<sup>2</sup>, PID<sup>1b</sup>)
5. participates in (GameDate<sup>2</sup>, TeamName<sup>3</sup>)

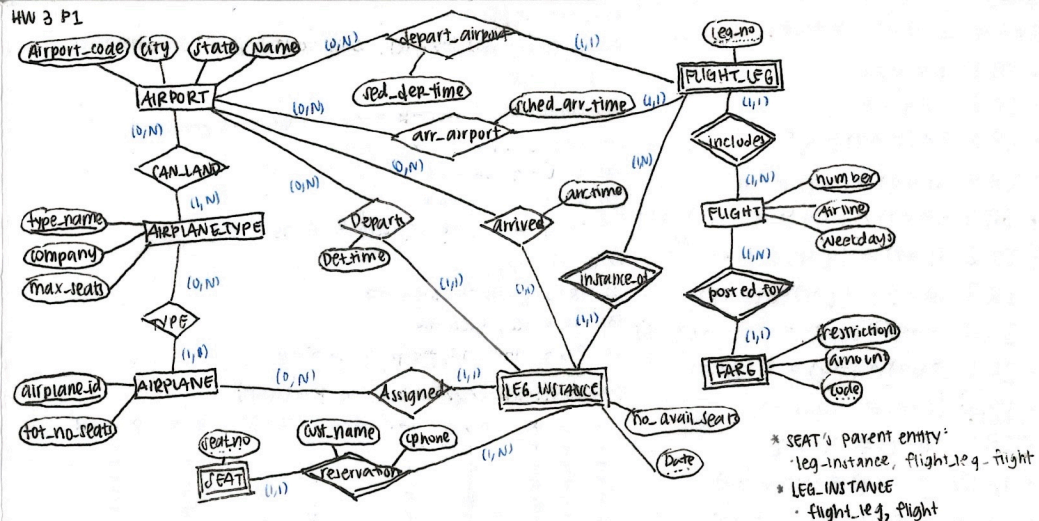
① Strong entities relation → ② weak entities relation + FK → ③ 1:1 total → new partial → add to total  
 ④ 1:N add to N-side → ⑤ M:N new relation, parents' PK:PK → ⑥ multivalued new relation

## HW 2



1. LIBRARY-MEMBER (SSN, phone, campus, address)
  - 1a. FACULTY (SSN<sup>1</sup>)
  - 1b. NON-FACULTY (SSN<sup>1</sup>)
2. STAFF-MEMBER (SSN)
  - 2a. CHIEF (SSN<sup>2</sup>)
  - 2b. DEPT-ASOC (SSN<sup>2</sup>)
  - 2c. REFERENCE (SSN<sup>2</sup>)
  - 2d. ASSISTANT (SSN<sup>2</sup>)
  - 2e. CHECKOUT-STAFF (SSN<sup>2</sup>)
3. BOOK (ISBN, title, author, lang, binding)
  - 3a. BOOK-TO-ACQUISITION (ISBN<sup>1</sup>)
  - 3b. BOOK-IN-LIB (ISBN<sup>1</sup>)
  - 3ba. BOOK-CAN-BE-LOANED (ISBN<sup>1</sup>)
  - 3bb. BOOK-CANNOT-BE-LOANED (ISBN<sup>1</sup>)
4. BOOK-COPY (book-copy-no, ISBN<sup>3b</sup>, checkout-status)
  - 4a. ISSUES-NOTICE-TO (staff-member-SSN<sup>2</sup>, library-member-SSN<sup>1</sup>, date)
  - 4b. ISSUES-CARD-TO (staff-member-SSN<sup>2</sup>, lib-member-SSN<sup>1</sup>, card-no, date, photo)
  - 4c. SUBJECT-AREA (book-ISBN<sup>3</sup>, subject-area-name)
  - 4d. CHECKS-OUT (lib-member-SSN<sup>1</sup>, book-copy-no<sup>4</sup>, book-ISBN<sup>3b</sup>, due-date)
5. RETURNS (lib-member-SSN<sup>1</sup>, book-copy-no<sup>4</sup>, book-ISBN<sup>3b</sup>, return-date)

## HW 3 P1



\* SEAT's parent entity:  
 - leg-instance, flight-leg-flight  
 \* LEG-INSTANCE  
 - flight-leg, flight

1. AIRPORT (airport-code, city, state, name)
2. AIRPLANE-TYPE (airplane-type-name, company, max-seats)
3. AIRPLANE (airplane-id, tot-no-seats, airplane-type-name<sup>2</sup>)
4. FLIGHT (flight-number, airline, weekdays)
5. FARE (code, flight-number<sup>4</sup>, amount, restrictions)
6. FLIGHT-LEG (leg-no, flight-number<sup>4</sup>, dep-airport-code<sup>1</sup>, dep-time, arr-airport-code<sup>1</sup>, arr-time)
7. LEG-INSTANCE (date, leg-number<sup>4</sup>, flight-number<sup>4</sup>, no-avail-seats, dep-airport-code<sup>1</sup>, dep-time, arr-airport-code<sup>1</sup>, arr-time, airplane-id<sup>3</sup>)
8. SEAT (seat-no, date<sup>7</sup>, leg-no<sup>4</sup>, flight-number<sup>4</sup>, cust-name, phone)
9. CAN-LAND (airplane-type-name<sup>2</sup>, airport-code<sup>1</sup>)



# ① Inference Rules $\models$ true in all cases

- IR 1:  $P \models P \vee Q$
- IR 2:  $P \wedge Q \models P$
- IR 3:  $P \wedge (P \rightarrow Q) \models Q$
- IR 4:  $(P \rightarrow Q) \wedge (\neg Q) \models \neg P$
- IR 5:  $(P \vee Q) \wedge (\neg P) \models Q$  ( $P \vee Q \wedge (\neg Q)$ )
- IR 6:  $(P \rightarrow Q) \wedge (Q \rightarrow R) \models P \rightarrow R$
- IR 7:  $(P \rightarrow Q) \models (Q \rightarrow R) \rightarrow (P \rightarrow R)$
- IR 8:  $(P \rightarrow Q) \wedge (R \rightarrow S) \models P \wedge R \rightarrow (Q \rightarrow S)$
- IR 9:  $(P \vee Q) \wedge (\neg Q) \models P$  (IR 5)
- IR 10:  $(P \rightarrow Q) \equiv \neg Q \rightarrow \neg P$
- IR 11:  $(P \rightarrow Q) \equiv \neg P \vee Q$
- IR 12:  $\neg(P \wedge Q) \equiv (\neg P \vee \neg Q)$
- IR 13:  $\neg(P \vee Q) \equiv (\neg P \wedge \neg Q)$

## HW2 Q1

$\neg(P \vee \neg Q) \vee (\neg P \wedge \neg Q) \equiv \neg P$

- assume  $\neg P$
- $\neg(P \vee \neg Q) \equiv (\neg P \wedge Q)$  IR13
- $(\neg P \wedge Q) \vee (\neg P \wedge \neg Q)$

It is required that  $\neg P$  &  $\neg Q/Q$  doesn't matter

$P \quad Q \quad \neg P \quad \neg Q \dots$  truth table

$P \vee \neg Q \equiv (P \rightarrow R) \wedge (Q \rightarrow R)$   
true (from truth table)

$\{t \rightarrow p, \neg p \vee s\} \models \neg s \rightarrow \neg t$

- assume  $\neg s$ , then  $\neg p$  must be true for  $\neg p \vee s$  to be true

IR 9:  $(\neg p \vee s) \wedge (\neg s) \models \neg p$

- if  $\neg p$  true, then  $p$  is false and  $t$  is false for  $t \rightarrow p$  to be true

IR 4:  $(t \rightarrow p) \wedge (\neg p) \models \neg t$

- $\neg s \rightarrow \neg t$

## Common Table

| P | Q | $\neg P$ | $\neg Q$ | $P \wedge Q$ | $P \vee Q$ | $P \rightarrow Q$ |
|---|---|----------|----------|--------------|------------|-------------------|
| T | T | F        | F        | T            | T          | T                 |
| T | F | F        | T        | F            | T          | F                 |
| F | T | T        | F        | F            | T          | T                 |
| F | F | T        | T        | F            | F          | T                 |

$(P \vee Q) \rightarrow R \equiv (P \rightarrow R) \wedge (Q \rightarrow R)$

$(P \vee Q) \rightarrow (P \wedge Q) \rightarrow \text{False}$

|   |   |   |
|---|---|---|
| T | T | T |
| T | T | F |
| T | F | F |
| F | T | F |
| F | F | F |

$p=T, q=F$   
 $p \vee q = T$ , but  
 $p \wedge q = F$ , so  $T \rightarrow F$  false

## PRAC MT Q1 (all True)

Given  $(P \rightarrow Q)$  and  $(\neg P \rightarrow Q)$ ,  $Q$  is tautological consequence

IR3, True

The premises  $P \rightarrow Q; \neg R \rightarrow \neg Q; \neg R \vee S; \neg S$  entail  $\neg P$

(1) assume  $\neg S \rightarrow \neg P$ , then  $S \rightarrow P$

(2) if  $P \rightarrow Q$ , then  $Q$

(3) if  $\neg R \rightarrow \neg Q$ , then  $R$

(4)  $\neg R \vee S \Leftrightarrow S$  Since we have  $R \therefore \neg S \rightarrow \neg P$

(1) if  $\neg S \rightarrow \neg P$ ,

(2)  $\neg R \vee S$  given  $\neg S$  then  $\neg R$

(3)  $\neg R \rightarrow \neg Q$ , then  $\neg Q$

(4)  $P \rightarrow Q$ , then  $\neg Q \rightarrow \neg P \therefore \neg S \rightarrow \neg P$

$(P \rightarrow Q) \vee (P \rightarrow R)$  equivalent to  $P \rightarrow (Q \vee R)$

| P | Q | R | $P \rightarrow Q$ | $P \rightarrow R$ | $(P \rightarrow Q) \vee (P \rightarrow R)$ | $Q \vee R$ | $P \rightarrow Q \vee R$ |
|---|---|---|-------------------|-------------------|--------------------------------------------|------------|--------------------------|
| T | T | T | T                 | T                 | T                                          | T          | T                        |
| T | T | F | T                 | F                 | T                                          | T          | T                        |
| T | F | T | F                 | T                 | T                                          | T          | T                        |
| T | F | F | F                 | F                 | F                                          | F          | F                        |
| F | T | T | T                 | T                 | T                                          | T          | T                        |
| F | T | F | T                 | T                 | T                                          | T          | T                        |
| F | F | T | T                 | T                 | T                                          | T          | T                        |
| F | F | F | T                 | T                 | T                                          | F          | T                        |

(1) assume  $P$ , then  $Q$

(2) assume  $P$ , then  $R$

(3)  $Q \vee R$

(4)  $P \rightarrow (Q \vee R)$  True

$(P \rightarrow (Q \rightarrow R))$  and  $((P \wedge Q) \rightarrow R)$  equivalent

\* note that  $F \wedge F$  is  $F$

\* truth table

$(P \vee Q) \rightarrow R$  and  $(P \rightarrow R) \wedge (Q \rightarrow R)$  equivalent

\* truth table

$(P \vee Q) \rightarrow R \equiv \neg(P \vee Q) \vee R$  IR12

$\equiv (\neg P \wedge \neg Q) \vee R$  IR13

$\equiv (R \vee \neg P) \wedge (R \vee \neg Q)$  Distrib.

$\equiv (P \rightarrow R) \wedge (Q \rightarrow R)$  IR12

| P | Q | R | $P \wedge Q$ | $P \vee Q$ | $\neg R$ | $P \rightarrow R$ | $(P \wedge Q) \rightarrow R$ | $Q \rightarrow R$ |
|---|---|---|--------------|------------|----------|-------------------|------------------------------|-------------------|
| T | T | T | T            | T          | F        | T                 | T                            | T                 |
| T | T | F | T            | T          | T        | F                 | F                            | F                 |
| T | F | T | F            | T          | F        | T                 | F                            | T                 |
| T | F | F | F            | T          | T        | T                 | T                            | T                 |
| F | T | T | F            | T          | F        | T                 | T                            | T                 |
| F | T | F | F            | T          | T        | T                 | T                            | T                 |
| F | F | T | F            | F          | F        | T                 | T                            | T                 |
| F | F | F | F            | F          | T        | T                 | T                            | T                 |

$P \rightarrow Q \equiv \neg P \vee Q$

$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$

$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$

$(P \vee Q) \rightarrow R \equiv (P \rightarrow R) \wedge (Q \rightarrow R)$

$P \rightarrow (Q \rightarrow R) \equiv (P \wedge Q) \rightarrow R$

$(P \rightarrow Q) \wedge (\neg P \rightarrow Q) \equiv Q$

$(P \vee Q) \rightarrow R \equiv (\neg P \wedge \neg Q) \vee R$

## ② EER $\rightarrow$ Relational Schema

### 1) Strong entities

- create relation
  - single-value attributes
  - primary key (PK) = entity's key underlined
  - composite attributes = split into simple attributes

### 2) weak entities

- create relation like strong, plus...
  - add foreign key (FK) referencing parent entity
  - PK = partial key + FK
  - if owner is weak, include all PK attributes

### 3) 1:1 Relationships

- total participation  $\rightarrow \min=1$
- partial participation  $\rightarrow \min=0$
- if both sides total:
  - merge/create relation for entities & relationship
- if one side total:
  - add other side's PK as FK to total side
  - FK as PK or unique key
- neither side total:
  - use multiple to multiple relationship

### 4) 1:N Relationships

- add foreign key to N-side, referencing 1-side's PK
- include relationship attributes in N-side

### 5) M:N Relationships

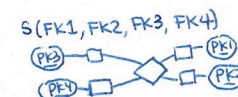
- create new relation
- both entities' PK as FK (to form new PK)
- add relationship attributes

### 6) Multivalued Attributes (double circle)

- create new relation
- include owner's PK as FK + multivalued attr. itself
- PK = (owner PK + attribute)

### 7) N-Ary Relationships

- create new table w/ all entities' PKs
- add relationship attributes



### 8) Sub/superclass

- superclass w/ own PK
  - subclass own relation w/ superclass PK
  - subclass specific attributes
- works for all disjoint/overlapping, total/partial